



Packet Scheduling

Purpose of this lab:

Step 1: Packet scheduling algorithms determine the order of packet transmission at the output link of a packet switch. This lab includes experiments that exhibit properties of well-known scheduling algorithms, such as is FIFO (First-in-First-out), SP (Static Priority), and DRR (Deficit Round Robin).

Software Tools:

- This lab uses traffic traces from Lab 1 and traffic generator and sink components from Lab 2.
- Source code of a reference implementation for a packet scheduler is provided.

What to turn in:

- On Crowdmark: A report with your answers to the questions in this lab, including the plots.
- On Quercus: Your modified code files.

February 2026

Deleted: Introduction to the Internet Lab

© Jörg Liebeherr, 2007-2026. All rights reserved. Permission to use all or portions of this material for educational purposes is granted, as long as use of this material is acknowledged in all derivative works.

Table of Content

COMMENTS	2
THE PACKET SCHEDULER CLASS	3
PART 1. FIFO SCHEDULING	6
<i>Exercise 1-a. Traffic generator for compound Poisson traffic</i>	6
<i>Exercise 1-b. Implement a FIFO Scheduler</i>	7
<i>Exercise 1-c. Observing a FIFO Scheduler at different loads</i>	7
<i>Exercise 1-d. (Optional, 5% extra credit) Unfairness in FIFO</i>	7
PART 2. PRIORITY SCHEDULING	9
<i>Exercise 2-a. Transmission of tagged packets</i>	10
<i>Exercise 2-b. Packet classification and priority scheduling</i>	11
<i>Exercise 2-c. Evaluation of the priority scheduler</i>	11
<i>Exercise 2-d. (Optional, 5% extra credit) Starvation in priority schedulers</i>	12
PART 3. DEFICIT ROUND ROBIN (DRR) SCHEDULER	13
<i>Exercise 3-a. Building a DRR scheduler</i>	14
<i>Exercise 3-b. Evaluation of a DRR scheduler: Equal weights</i>	15
<i>Exercise 3-c. Evaluation of a DRR scheduler: Different weights</i>	15
<i>Exercise 3-d. (Optional, 5% extra credit) No Unfairness and no Starvation in DRR</i>	15
FEEDBACK FORM FOR LAB 3	17

Comments

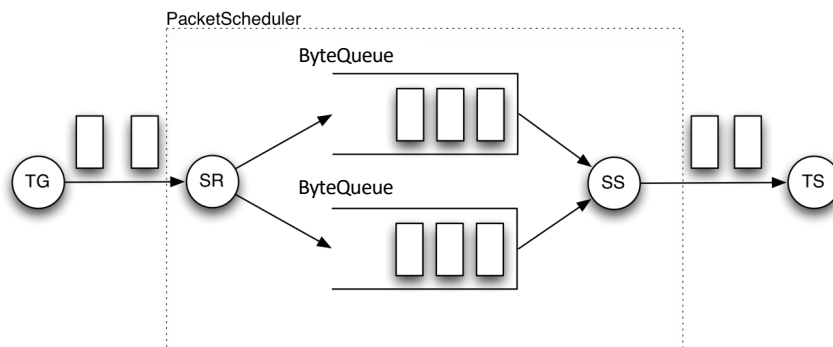
Extra credit: Each part of this lab has an extra credit component (5% each).

Provided python code: We provide python code for the building blocks of a packet scheduler. The files are contained in the ZIP file *scheduler.zip*, which is available on the Quercus course page. The ZIP file contains the following python programs:

byte_queue.py
scheduler_receiver.py
packet_scheduler.py
scheduler_sender.py

The Packet Scheduler class

In this lab, you will implement three different packet scheduling algorithms. All packet schedulers can be implemented by extending the provided PacketScheduler code. There are many similarities between the PacketScheduler class and the TokenBucket class from Labs 2a/2b.



In essence, a PacketScheduler object receives packets from one or more traffic generators (TG), and forwards packet to a single traffic sink (TS).

A PacketScheduler object has three components: a scheduler receiver (SR), a scheduler sender (SS), and one or more FIFO Buffers. The Scheduler Receiver SR processes incoming packets sent by the traffic generator(s), and adds the packet to one of the FIFO buffers. The scheduler inspects the first byte of each arriving packet to perform a **traffic classification**, which determines the FIFO buffer to which the packet must be added.

Important: Traffic classification assumes that the traffic generator sets the first byte of the packet to the correct value. We refer to this process as **traffic tagging**. Therefore, previously used traffic generators must be modified so that they include a traffic classifier.

For example, for a priority scheduler, the traffic generators must tag each packet as either “High” or “Low” priority. When SR reads the tag, it can classify a packet as “High” or “Low” priority and add the packet to the appropriate FIFO buffer.

The following code segment shows an example of setting and reading the first byte of data from a packet.

```
# Create mutable payload
payload = bytearray(length)
# Set the first byte to 1
payload[0] = 0x01; # must be 0-255

# Send the payload to destPort at destAddr
socket.sendto(payload, (destAddr, destPort))

# Read the first byte from packet
value = payload[0];
```

The component SS transmits packets that are backlogged in one of the buffers. The key task of SS is the **scheduling decision**, which determines the FIFO buffer from which the next packet is transmitted. Once the decision is made, the first packet from the selected FIFO buffer is removed and transmitted on the link, at the specified transmission rate.

The reference implementation of the Packet Scheduler implements the above functions, however, some components are incomplete:

- In the reference implementation, SR does not perform traffic classification. Instead, it adds packets to the first of the available FIFO buffers (the buffer with the smallest index).
- The scheduling decision is simplified, in that the reference implementation always picks a packet from the first FIFO buffer.
- The recording of arrival times in a file does not include information from the traffic classification (that is, it would not contain priority information for a priority scheduler).

The implementation is provided by the package PacketScheduler, which consists of three Python classes:

- **PacketScheduler**

Creates the components of the packet scheduler, and starts threads that run in SR and SS. An instant of PacketScheduler is created with the following parameters:

- `in_port` – UDP Port number, where SR expects arriving packets.
- `out_ip` – IP address to which SS sends packets.
- `out_port` – UDP port number to which SS sends packets.
- `link_capacity` – transmission rate of the scheduler (in bits per second).
- `max_packet_size` – Maximum size of packet that can be processed. (in bytes)
- `buffer_capacities[]` – Array containing the capacities of the FIFO buffers (in bytes).
- `logfile` – Name of file, where arrival times are recorded.

- **ByteQueue**

This is similar to the FIFO Buffer that was used for TokenBucket, with additional Semaphore called `nonempty` to store the number of non-empty buffers. PacketScheduler creates an array of buffers, named `buffers`. The implementation of the FIFO buffers is thread safe. The buffers store packets. The capacity of `buffers[i]` is given by `buffer_capacities[i]`, specified in bytes. Each buffer has methods for adding packets to the tail of the buffer and removing packets from the head of the buffer, peeking at the first packet (without removing it), querying the currently occupied buffer size (in terms of bytes), and determine whether the buffer is empty. When adding a newly arriving packet would exceed `buffer_capacities[i]`, the packet is dropped.

- **SchedulerReceiver (SR)**

Waits on a specified UDP port for incoming packets. When a packet arrives, the packet is first classified by inspecting the first byte of the packet. The content of the byte is mapped to an index of the FIFO buffer. The mapping depends on the type of scheduler used. Once classified, SR records the arrival time, size, buffer backlog of the queue, to a file. Then the packet is added to `buffers[index]`:

Traffic classification:

```
class = {content of first byte};  
{Map class to index of the buffer array};  
add packet to buffers[index];
```

If a packet cannot be added to the buffer, e.g., because the buffer is full, the packet is dropped and an error message displayed.

- **SchedulerSender (SS)**

SchedulerSender (SS) is responsible for selecting the buffer from which the next packet transmission occurs. This is referred to as the scheduling decision. The scheduling decision is dependent on the type of scheduling algorithm. The scheduler removes the first packet from the selected buffer and transmits the packet to the specified address (IP address and UDP port). The time to transmit a packet with size L bits is computed to be $L/Link_capacity$. If the next packet arrives when the previous packet has yet to complete its transmission, SS will sleep for the remaining time. The procedure for transmission is as follows:

```
wait for at least one buffer to be non-empty;  
# buffers wake up SS when a packet arrives via the shared `nonempty` Semaphore  
  
determine buffer from which packet is transmitted;  
remove first packet from selected buffer;  
  
if (the previous packet has not completed its transmission)  
    sleep/wait until the transmission is completed;  
  
transmit packet;  
computed the time when this packet completes its transmission;
```

The PacketScheduler of the reference implementation is started with the command:

```
python3 packet_scheduler <inPort> <outIP> <outPort> <C> <buf0Size> <buf1Size> ...
```

where

inPort	– port number where PacketScheduler listens to arrivals
outIP, outPort	– IP address and port number of the traffic sink
buf0Size, buf1Size, ...	– buffer capacity of each buffer
C	– rate at which the sender transmits packets to traffic sink

This command starts a packet scheduler, which listens on <inPort>, and forwards packets to <outIP> at <outPort>.

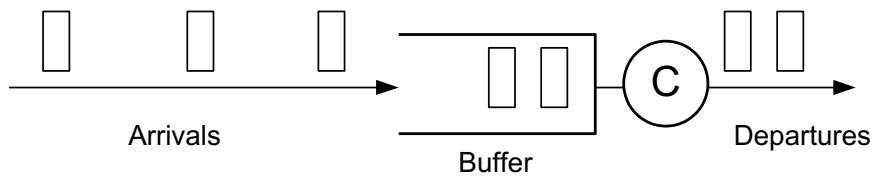
You may add the following optional arguments:

```
--max-packet-size: maximum packet size in bytes (default: 1480 Bytes)  
--logfile: the name of the log file to store the arrival time (default: arrivals.log)
```

Part 1. FIFO Scheduling

In this part, you explore the backlog and delay at a link with a FIFO (First-In-First-Out) buffer, when the traffic load arriving to the link is increased.

The FIFO buffer operates at link capacity $C=10$ Mbps. The situation is illustrated in the figure, where packets are represented as rectangular boxes. With FIFO, also referred to as FCFS (First-Come-First-Served), packets are transmitted in the order of their arrival.



The objective is to observe and measure the backlog and delay at a FIFO buffer when the load is varied. For traffic arrivals you will use the compound Poisson process from Lab 1.

Exercise 1-a. Traffic generator for compound Poisson traffic

Work with the traffic generator from Part 2 of Lab 2a, which is based on the compound Poisson arrival process from Lab 1, where packet arrival events follow a Poisson process with rate $\lambda = 1250$ packets/sec, and the packet size has an exponential distribution with average size $1/\mu = 100$ Bytes. The average generated traffic rate is 1 Mbps.

Step 1: Download a trace file *poisson3.data* from the Quercus page.

Step 2: Consider the traffic generator from Part 2 of Lab 2a.

At a link with $C=10$ Mbps, the Poisson source creates an average load of 10% of the link capacity. The load of a link, also referred to as utilization and denoted by ρ , indicates the percentage of time that a work-conserving link will be busy (busy = transmitting a packet). The utilization is computed as follows:

$$\begin{aligned}\rho &= (\text{Average packet arrival rate}) \times (\text{average transmission time of packet}) \\ &= \lambda \times 1/(\mu C)\end{aligned}$$

Step 3: Add a feature to the code of your traffic generator that can re-scale the time between packet transmissions to generate an average traffic rate of N Mbps, where $N = 1$ (low load), $N=5$ (medium load), $N=9$ (high load). For example, double the average rate of generate traffic, the distance between subsequent packets should be half of the distance given in the file.

Step 4: Test the correctness of the traffic generator, using the traffic sink from Exercise 2-b in Lab 2a.

Exercise 1-b. Implement a FIFO Scheduler

Step 1: Use the reference implementation of PacketScheduler to build a FIFO scheduler. The FIFO scheduler transmits packets in the order of their arrivals.

- Set the maximum size of the buffer in the FIFO scheduler to 100 kB. If the available buffer size is too small for an arriving packet, the packet is discarded (and a message is displayed.)
- Set the capacity of the link to 10 Mbps.

Note:

- With FIFO, the scheduler only needs a single FIFO buffer.
- There is no need for traffic tagging (by the traffic generator) or traffic classification (by PacketReceiver).

Exercise 1-c. Observing a FIFO Scheduler at different loads

Use the traffic generator to evaluate the FIFO scheduler with compound Poisson traffic at different loads.

Step 1: Use the added feature in the traffic generator from Exercise 1-a and run the re-scaled Poisson trace file with an average rate of N Mbps, where $N = 1$ (low load), $N = 5$ (medium load), $N = 9$ (high load).

For each value of N :

- Compute the backlog and waiting time for each packet processed by the packet scheduler.
Note: The backlog can be determined when a packet arrives. Determining the waiting time requires that you know the arrival time and the departure time of a packet.
- Keep track of arriving packets that are discarded.

Step 2: For low, medium and high load scenarios, generate plots that show the backlog, waiting time, and number of discarded packets, as a function of time.

Step 3: Designate ranges of N , where the FIFO scheduler is in a regime of low load and high load. Justify your choice.

Exercise 1-d. (Optional, 5% extra credit) Unfairness in FIFO

A limitation of FIFO is that it cannot distinguish different traffic sources. Suppose that there are many low-bandwidth sources and a single traffic source that sends data at a very high rate. If the high-bandwidth source causes an overload at the link, then all traffic sources will experience packet losses due to buffer overflows. From the perspective of a low-bandwidth source, this seems unfair. (The low-bandwidth source would like to see that packet losses experienced at the buffer are proportional to the traffic rate of a source).

Here you create traffic scenarios that exhibit the unfairness issues of FIFO for two traffic sources. There are two traffic sources, which are each re-scaled compound Poisson sources as in Exercise 1-a:

- Source 1 sends at an average rate of N_1 Mbps.
- Source 2 sends at an average rate of N_2 Mbps.

Both sources send traffic into a FIFO scheduler (with rate $C=1$ Mbps) and 100 kB of buffer.

Step 1: Create a transmission scenario with two traffic generators, one for each traffic source that send traffic into a FIFO scheduler. The FIFO scheduler forwards packets to a single traffic sink.

The traffic sink must be able to determine if a packet was sent by Source 1 or Source 2. This can be done by tagging the packets at the traffic generators.

Step 2: Run a series of experiments where the load of the two sources is set to (N_1, N_2) with $N_1 = 5$ and $N_2 = 1, 5, 10$. At the traffic sink, record the average throughput (output rate) of each traffic source.

Step 3: Prepare a table that shows the average throughput values and interpret the result. Can you devise a formula that predicts the throughput of a source as a function of the arrival rate?



Lab Report

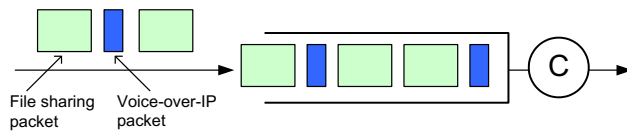
Provide the plots and a discussion of the plots. Also include answers to the questions.

Part 2. Priority Scheduling

FIFO scheduling gives all traffic the same type of service. If the network carries traffic types with different characteristics and service requirements, a FIFO scheduling algorithm is not sufficient. To differentiate traffic types and give each type a different grade of service, more sophisticated scheduling algorithms are needed. One of these algorithms is the priority scheduling algorithm.

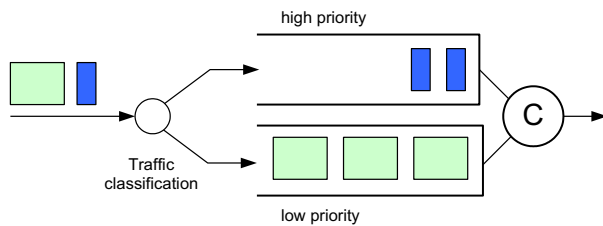
For example, consider a mix of file sharing traffic and voice (e.g., voice-over-IP) traffic: File sharing traffic is high-volume and transmitted in large packets, whereas voice-over-IP traffic has a relatively low data rate and is transmitted in short packets. If the traffic is handled by a FIFO scheduler, as shown below, then voice packets may experience a poor grade of service dependent on the on the arrivals of file sharing packets.

FIFO scheduler:



A priority scheduler can improve the service given to voice packets by giving voice packets higher priority. A priority scheduler always selects the packet with the highest priority level for transmission.

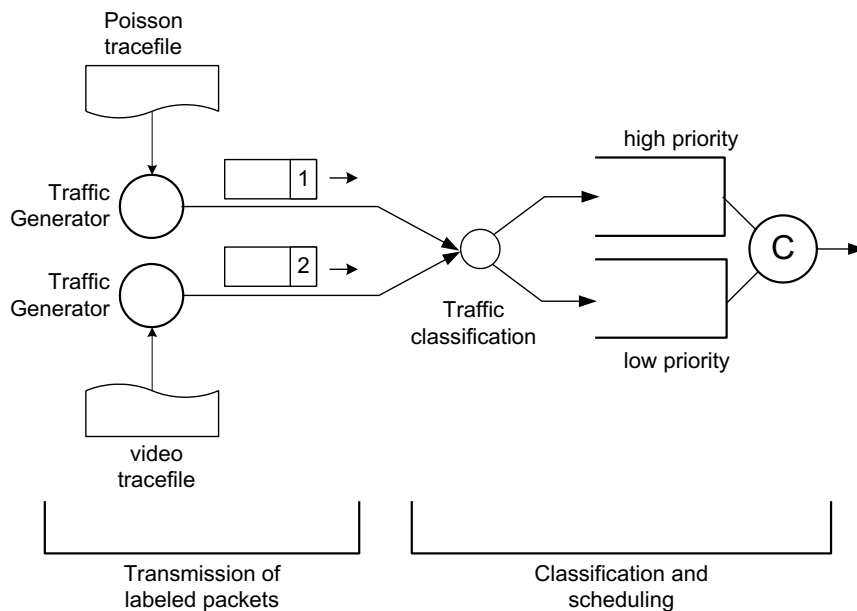
Priority scheduler:



A priority scheduler assumes that incoming traffic can be mapped to a priority level. A traffic classification component of the scheduler uses an identifier in incoming packets to perform this mapping. The identifier can be based on source and destination addresses, application type application type (e.g., via the port number), or other information in packet headers.

Priority schedulers are also referred to as static priority (SP) or Head-of-Line (HOL) schedulers.

In this part of the lab, you will design and implement a priority scheduler with two priority levels, as shown in the Figure below. Traffic is transmitted to the scheduler from two sources: a compound Poisson source and a video source. The sources mark packets with a tag: "1" for packets from the Poisson source and "2" for packets from the video source. A traffic classifier at the priority scheduler reads the identifier. Packets with tag "1" are handled as low priority, and packets with tag "2" are handled as high priority packets.



Exercise 2-a. Transmission of tagged packets

Step 1: Build a pair of traffic generators as shown on the left-hand side of the figure above. One traffic generator transmits Poisson traffic the other traffic generator generates video traffic. The transmissions of the video source and the Poisson source are performed by two distinct programs.

- The tracefile for the Poisson is *poisson3.data* from Part 1.
- The tracefile for the video traffic is *movietrace.data*.

The tracefiles are available on Quercus in the materials for Lab 1.

The average rate of traffic from the video source is approx. 15 Mbps.

- A traffic generator for the video source can be built by re-using the code from Part 2 of Lab 2a. As in Lab 2a, the amount of data that can be put into a single packet is limited to 1480 bytes. Frames exceeding this length are divided and transmitted in multiple packets.
- The transmission of the Poisson source is determined by the Poisson traffic generator built in Exercise 1-a of this lab (Lab 3). The traffic generator must be able to run the re-scaled Poisson trace file with an average rate of N Mbps, where $N = 1$ (low load), $N = 5$ (medium load), $N = 9$ (high load).
- Before a packet is transmitted it must be tagged with an identifier, which is located in the first byte of the payload. The identifier is the number 0x01 for packets from the Poisson source and 0x02 for the video source.

- Packets are transmitted to a remote UDP port. Both sources transmit UDP datagrams to the same destination port on the same host (e.g., port 4444). You may use a traffic sink as build for Exercise 2-b of Lab 2a for testing the implementation.

Exercise 2-b. Packet classification and priority scheduling

Step 1: Build a traffic classification and scheduling component as shown on the right hand side of the above figure.

- The priority scheduler consists of two FIFO queues: one FIFO queue for high priority traffic and one FIFO queue for low priority traffic. Set the maximum buffer size of each FIFO queue to 100 kB.
- The priority scheduler always transmits a high-priority packet, if the high priority FIFO queue contains a packet. Low priority packets are selected for transmission only when there are no high priority packets.
- The transmission rate of the link is **C=20 Mbps**.
- The traffic classification component reads the first byte of the payload of an arriving packet and identifies the priority label.
- Once classified, packets are assigned to the priority queues. Video traffic (with label "2") is assigned to the high priority queue and Poisson traffic (with label "1") is assigned to the low priority queue.
- If a new packet arrives when the link is idle (i.e., no packet is in transmission and no packet is waiting in the FIFO queues) the arriving packet is immediately transmitted. Otherwise, the packet is enqueued in the corresponding FIFO queue.
- The priority scheduler is work-conserving: As long as there is a packet waiting, the scheduler must transmit a packet.
- Packet transmissions are non-preemptive: Once the transmission of a packet has started, the transmission cannot be interrupted. In particular, when a low priority packet is in transmission, an arriving high-priority packet must wait until the transmission is completed.

Step 2: Test the implementation with the traffic generator from Exercise 2-a.

Exercise 2-c. Evaluation of the priority scheduler

Step 1: Evaluate the priority scheduler with the traffic generator Exercise 2-a using the following transmission scenarios:

- The video source transmits according to the data in the tracefile (see Exercise 2-a).
- As in Exercise 1.1, the Poisson source should be scaled so that it transmits with an average rate of N Mbps, where N = 1 (low load), N=5 (medium load), N=9 (high load).
- For each value of N, determine the following values for both high priority (video) and low priority (Poisson) traffic:

- Compute the backlog and waiting time for each packet processed by the packet scheduler.
Note: The backlog can be determined when a packet arrives. Determining the waiting time requires that you know the arrival time and the departure time of a packet.
- Keep track of arriving packets that are discarded.

Step 2: Create graphs that show the backlog, waiting time, and number of discarded packets, as a function of time.

Step 3: Compare the outcome to Exercise 1-c.

Exercise 2-d. (Optional, 5% extra credit) Starvation in priority schedulers

A limitation of SP scheduling is that it always gives preference to high-priority scheduling. If the load from high-priority traffic is very high, it may completely pre-empt low-priority traffic from the link. This is referred to as *starvation*.

This exercise exhibits the starvation of low priority traffic. You proceed similarly as in the last exercise of Part 1.

Step 1: Create a transmission scenario with two traffic generators, an SP scheduler, and a traffic sink.

- The link rate of the SP scheduler is **C=10 Mbps**. The SP scheduler has two FIFO buffers that each have a limit of 100 kB.
- There are two traffic sources, which are each re-scaled compound Poisson sources as in Exercise 1-a:
 - Source 1 transmits at an average rate of N_1 Mbps.
 - Source 2 transmits at an average rate of N_2 Mbps.
 - Traffic from Source 1 is labelled with identifier “1” (low priority) and Source 2 is labelled with “2” (high priority).

Step 2: Run a series of experiments where the load of the two sources is set to (N_1, N_2) with $N_1 = 5$ and $N_2 = 1, 5$, and 9.

- Record the average throughput (output rate) of each traffic source.
- Prepare a table that shows the average throughput of high and low priority traffic and interpret the result.
- Compare the outcome to Exercise 1-d.

Lab Report



- Provide the created tables and graphs, with brief discussions. Also include answers to the questions in the lab exercises.

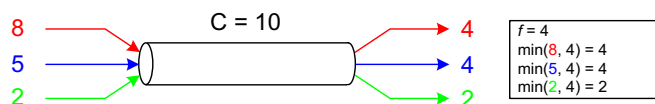
Part 3. Deficit Round Robin (DRR) Scheduler

Many scheduling algorithms attempt to achieve a notion of **fairness** by regulating the fraction of link bandwidth allocated to each traffic source. The objectives of a “fair scheduler” are as follows: If the link is not overloaded, a traffic source should be able to transmit all of its traffic. If the link is overloaded, each traffic source obtains the same rate guarantee, called the *fair share*, with the following rules:

- If the traffic from a source is less than the fair share, it can transmit all its traffic;
- If the traffic from a source exceeds the fair share, it can transmit at a rate equal to the fair share.

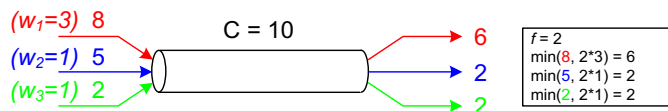
The fair share depends on the number of active sources and their traffic rate. Suppose we have set of sources where the arrival rate of Source i is r_i , and a link with capacity C (bps). If the arrival rate exceeds the capacity, i.e., $\sum_i r_i \geq C$, then the fair share is the number f that satisfies the equation $\sum_i \min(r_i, f) = C$.

As an example, suppose we have a link with capacity 10 Mbps, and the arrival rates of flows are $r_1=8$ Mbps, $r_2=6$ Mbps, and $r_3=2$ Mbps, then the fair share is $f=4$ Mbps, resulting in an allocated rate is 4 Mbps for Source 1, 4 Mbps for Source 2, and 2 Mbps for Source 3.



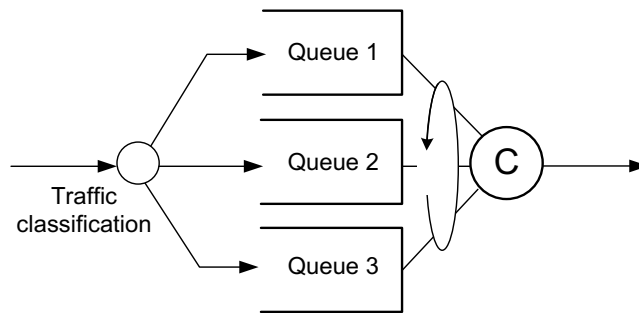
Since different sources have different resource requirements, it is often desirable to associate a weight (w_i) with each source, and allocate bandwidth proportionally to the weights. In other words, a source that has a weight twice as large of second source should be able to obtain twice the bandwidth of the second source. With these weights, the fair share f at an overloaded link, i.e., $\sum_i r_i \geq C$, is obtained by solving $\sum_i \min(r_i, f w_i) = C$.

For example, using the previous example, and assigning weights $w_1=3$, $w_2=w_3=1$, the allocated rates are 6, 2, and 2 Mbps for the three sources.



A scheduling algorithm that realizes this scheme without weights are called Fair Queueing (FQ) or Processor Sharing (PS). For scheduling with weights, the algorithm is called Weighted Fair Queueing (WFQ) or Generalized Processor Sharing (GPS). These algorithms are idealized algorithms, since they treat traffic as a fluid. Realizing fairness in a packet network turns out to be quite hard, since packet sizes have different sizes (50 – 1500 bytes) and packet transmissions cannot be interrupted. Many commercial IP routers and Ethernet switches implement scheduling algorithms that approximate fair queueing.

A widely used (and easy to implement) scheduling algorithm that approximates GPA is the Deficit Round Robin (DRR) scheduler. The objective of this part of the lab is implementing and evaluating a DRR scheduling algorithm.

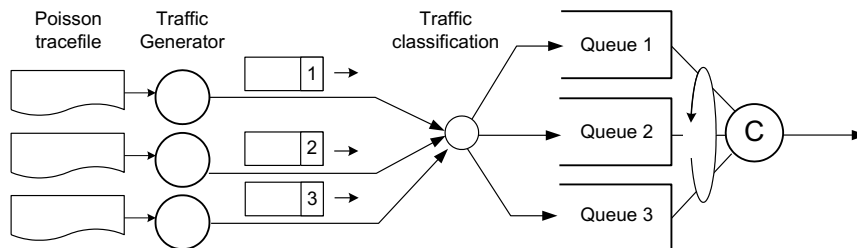


Details of DRR scheduling

The operation of the DRR scheduler is explained in detail in the slide deck “Variants of Fair Queueing.”

Exercise 3-a. Building a DRR scheduler

Step 1: Create a DRR scheduler as described above, that supports three queues. The DRR will serve Poisson traffic sources as used in Parts 1 and 2 of this lab.



- A DRR scheduler requires one FIFO buffer (queue) for each flow.
- There are three traffic generators that each transmit re-scaled Poisson traffic as done in Exercise 1-a. Each Poisson source is re-scaled so that it transmits with an average rate of N Mbps, where $N = 1$ (low load), $N=5$ (medium load), $N=9$ (high load).
- Each transmitted packet is labelled with “1”, “2” or “3” as done in Part 2 of this lab. The label is one byte long. The traffic classification component reads the first byte of the payload of an arriving packet, and adds the packet to the queue (Packets with label “1” are associated with Queue 1, etc.). If no packet is in transmission or in the queue, and arriving packet is transmitted immediately.

- The transmission rate of the link is $C=10$ Mbps. The maximum buffer size of each queue is 100 kB. An arrival that cannot be stored in the queue is discarded.

Once the implementation is completed and tested, move on to the evaluation.

Exercise 3-b. Evaluation of a DRR scheduler: Equal weights

Step 1: Evaluate the DRR scheduler with three Poisson sources.

- The weight N of the traffic generators are set to $N=8$ in for the first source, $N=6$ for the second source and $N=2$ for the third source, resulting in traffic arrival rates of 8 Mbps, 6 Mbps, and 2 Mbps.
- Set the weights of the queues to $w_1=w_2=w_3=1$.

Note: Compare this scenario to the first figure of Part 3. The average load on the link is 1.5 Mbps, i.e., the link is overloaded. We expect that the bandwidth at the link is shared among the sources at a ratio of 4:4:2.

Step 2: Prepare graphs that show the number of packet transmissions and the number of transmitted bytes from a particular source (y-axis) as a function of time (x-axis). Provide one plot for each source. Select a reasonable time scale for the x-axis, e.g., a time scale of 10 ms per data point.

Step 3: Compare the plots with the theoretically expected values of a PS scheduler.

Exercise 3-c. Evaluation of a DRR scheduler: Different weights

This exercise re-creates a transmission scenario as in the second figure of Part 3.

Step 1: Evaluate the DRR scheduler with three Poisson sources. As before, the weight N of the traffic generators is set to $N=8$ in for the first source, $N=6$ for the second source and $N=2$ for the third source. Set the weights of the queues to $w_1=3$ and $w_2=w_3=1$.

Note: Compare this scenario to the second figure of Part 3. We expect that the bandwidth at the link is shared among the sources at a ratio of 6:2:2.

Step 2: Prepare graphs that show the number of packet transmissions and the number of transmitted bytes from a particular source (y-axis) as a function of time (x-axis). Provide one plot for each source.

Step 3: Compare the graphs with the theoretically expected values of a GPS scheduler.

Exercise 3-d. (Optional, 5% extra credit) No Unfairness and no Starvation in DRR

A limitation of SP scheduling is that it always gives preference to high-priority scheduling. If the load from high-priority traffic is very high, it may completely pre-empt low-priority traffic from the link. This is referred to as *starvation*.

The following experiment tries to show that DRR does not suffer from the problems of FIFO (unfair) and SP (starvation). The experiment retraces the steps of Exercises 1-d and 2-d.

Step 1: Build a DRR scheduler as given above for two traffic classes. The link rate is $C=10$ Mbps and the buffer size of each queue 100 kB of buffer for each queue.

- There are two traffic sources, which are each re-scaled compound Poisson sources as in Exercise 1-a:
 - Source 1 transmits at an average rate of N_1 Mbps.
 - Source 2 transmits at an average rate of N_2 Mbps.
 - Traffic from Source 1 is labelled with identifier “1” and Source 2 is labelled with “2”.
 - Both sources are assigned the same weight at the DRR scheduler ($w_1=w_2=1$).

Step 2: Run a series of experiments where the load of the two sources is set to (N_1, N_2) with $N_1 = 5$ and $N_2 = 5, 9, 15$.

- Record the average throughput (output rate) of each traffic source.
- Prepare a table that shows the average throughput of the two sources and interpret the result.

Step 3: Compare the outcome to Exercises 1-d and 2-d.

Lab Report



Provide the graphs with brief discussions. Also include answers to the questions.

Feedback Form for Lab 3

The lab has been revised for the W2026 semester. We appreciate any feedback on any issues that you may have encountered, e.g.,

- Instructions that are in need of clarification;
- Issues with provided Python code;
- ...

If you have feedback, please add it to your lab report as an appendix.

Thank you!